SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Section / Batch:	2024 / AIML	**Date:**	

Code of Conduct

I P. MUNI KARTHIK / 192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."
Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking
Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.
Apply your algorithm to:
User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."
Your algorithm should maintain the following slots:
Departure City
Destination City
Travel Date
Number of Passengers
Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach

Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	3	3	3	3	16	85
2	4	3	3	3	3	18	
3	3	3	4	4	4	17	
4	3	4	3	3	3	17	
5	3	3	3	3	4	18	

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Reference Resolution Using Multiple Constraints

Given Discourse

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

1. Entities and Referring Expressions

Entities:

- Meena
- Kavya
- Laptop
- Project

Referring Expressions:

- she
- one
- She
- her
- it

2. Possible Antecedents

Referring Expression	Possible Antecedents
she	Meena, Kavya
one	Laptop
She	Meena, Kavya
her	Meena, Kavya
it	Laptop

3. Pseudocode

```
BEGIN

Input discourse

Identify all entities and pronouns

FOR each pronoun P
   Find possible antecedents
   Check gender agreement
   Check number agreement
   Check recency
   Check semantic compatibility
   Assign score to each candidate
   Select candidate with highest score
END FOR

Replace pronouns with selected antecedents

Output resolved discourse

END
```

4. Applying the Constraints

- **"she" → Kavya** because Kavya is the receiver of the laptop and "needed one for her project" is semantically appropriate.
- **"one" → laptop** because "one" refers to the object that was given.
- **"She" → Kavya** because Kavya is the most suitable recent female entity.
- **"her" → Meena** because Kavya thanked Meena.
- **"it" → laptop** because the object being used is the laptop.

5. Resolved Discourse

Meena gave Kavya a laptop because **Kavya** needed **the laptop** for **Kavya's** project. **Kavya** thanked **Meena** and started using **the laptop**.

6. How the Algorithm Avoids Incorrect Resolution

When multiple antecedents have similar grammatical properties, the algorithm uses multiple constraints instead of only grammatical agreement. **Recency** identifies the recent candidate, while **semantic compatibility** checks whether the candidate logically fits the action. The candidate with the highest combined score is selected.

Q2. Algorithm for Entity Chains and Discourse Coherence

Given Text

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

1. Entities and Referring Expressions

Entities:

- Anjali
- Laptop
- Processor
- Python
- Machine learning application

Referring Expressions:

- The laptop
- She
- it
- Anjali
- the computer

2. Entity Linking

Expression	Entity
Anjali	Anjali
The laptop	Laptop
She	Anjali
it	Laptop
Anjali	Anjali
the computer	Laptop

3. Entity Chains

Person Chain:

Anjali → She → Anjali

Laptop Chain:

Laptop → The laptop → it → the computer

4. Pseudocode

```
BEGIN

Input discourse

Divide text into sentences

Identify entities and referring expressions

FOR each referring expression
   Find possible matching entities
   Check gender, number and semantic similarity
   Link expression to the best matching entity
END FOR

Construct entity chains

Calculate entity continuity across sentences

IF important entities continue across sentences
   Mark discourse as coherent
ELSE
   Mark discourse as less coherent
END IF

Output entity chains and coherence result

END
```

5. Entity Continuity

There are 4 sentences.

- **Anjali** appears in sentences 1, 3 and 4.
- **Laptop** appears directly in sentence 1 and through “The laptop,” “it,” and “the computer” in later sentences.

Therefore, the main entities remain connected throughout the discourse.

6. Coherence Result

The discourse is coherent.

The sentences are logically connected because they continuously discuss **Anjali and her laptop**.

7. Effect of Breaking an Entity Chain

If “it” or “the computer” were incorrectly linked to another entity, the connection between the sentences would be lost. This would make the text confusing and reduce discourse coherence.

Q3. Algorithm for Identifying Discourse Relations

Given Discourse

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

1. Discourse Units

U1: The server crashed unexpectedly.

U2: As a result, users could not access the application.

U3: The technical team restarted the server.

U4: After that, the system became available again.

2. Discourse Markers

- **As a result** → Cause–Effect
- **After that** → Sequence

3. Pseudocode

```
BEGIN

Input discourse

Divide discourse into individual units

FOR each consecutive pair of units
   Identify discourse markers
   Analyze contextual meaning

   IF marker indicates consequence
      Assign Cause-Effect
   ELSE IF marker indicates order
      Assign Sequence
   ELSE IF one unit solves a problem
      Assign Problem-Solution
   ELSE IF one unit provides additional information
      Assign Elaboration
   END IF
END FOR

Construct discourse structure

Output identified relations

END
```

4. Discourse Relations

Units	**Relation**	**Explanation**
U1 → U2	Cause–Effect	Server crash caused users to lose access.
U2 → U3	Problem–Solution	Access problem leads to the team restarting the server.
U3 → U4	Sequence	After restarting, the system became available.

5. Discourse Structure

U1: Server crashed

↓ Cause

U2: Users could not access application

↓ Problem

U3: Team restarted server

↓ Sequence

U4: System became available

6. Justification

The discourse is logically coherent because each sentence is connected to the previous event. The server crash creates the problem, the technical team provides a solution, and the system becomes available afterward. The markers **"As a result"** and **"After that"** make the relationships explicit.

Q4. Algorithm for Dialogue Act Classification

Given Conversation

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."

1. Pseudocode

```
BEGIN

Input conversation

FOR each utterance
   Preprocess the utterance
   Extract keywords and linguistic features

   IF utterance contains greeting
      Act = Greeting

   ELSE IF utterance asks for information
      Act = Question

   ELSE IF utterance requests help
      Act = Request

   ELSE IF utterance provides information
      Act = Inform

   ELSE IF agent offers assistance
      Act = Offer

   ELSE IF utterance confirms information
      Act = Confirmation

   END IF

   Store dialogue act
END FOR
```

Generate dialogue-act sequence

Output classification

END

2. Classification

Speaker	Utterance	Dialogue Act
User	Hello, I need help with my assignment.	Greeting + Request
Agent	Sure, what problem are you facing?	Question
User	I do not understand machine learning.	Inform
Agent	I can explain the basic concepts to you.	Offer

3. Dialogue-Act Sequence

Greeting → Request → Question → Inform → Offer

4. Explanation

Dialogue-act recognition helps the conversational agent understand the user's communicative intention. For example, when the user makes a **Request**, the agent can ask a relevant **Question**. When the user provides information, the agent can generate an appropriate response or **Offer** assistance.
Thus, dialogue-act recognition improves the relevance and coherence of conversational responses.

Q5. Algorithm for Dialogue State Tracking

Given Conversation

User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."

1. Dialogue State

The system maintains the following slots:

- Departure City
- Destination City
- Travel Date
- Number of Passengers

2. Initial Dialogue State

Slot	Value
Departure City	Unknown
Destination City	Unknown
Travel Date	Unknown
Number of Passengers	Unknown

3. Pseudocode

BEGIN

Initialize all dialogue slots as EMPTY

WHILE required information is missing

```
    Receive user input

    Identify information from the input

    IF destination is found
        Update Destination City

    IF departure city is found
        Update Departure City

    IF travel date is found
        Update Travel Date

    IF passenger number is found
        Update Number of Passengers

    Find missing slots

    IF Destination City is missing
        Ask "Where would you like to travel?"

    ELSE IF Departure City is missing
        Ask "From which city?"

    ELSE IF Travel Date is missing
        Ask "What is your travel date?"

    ELSE IF Number of Passengers is missing
        Ask "How many passengers are travelling?"

    ELSE
        Confirm booking details
    END IF

END WHILE

END
```

4. Applying the Algorithm

Step 1:

User: "I want to book a flight."

State:

Slot	Value
Departure City	Unknown
Destination City	Unknown

Slot	Value
Travel Date	Unknown
Number of Passengers	Unknown

System asks:

"Where would you like to travel?"

Step 2:

User: "I want to go to Delhi."

Update:

Destination City = Delhi

System asks:

"From which city?"

Step 3:

User: "From Chennai."

Update:

Departure City = Chennai

5. Current Dialogue State

Slot	Value
Departure City	Chennai
Destination City	Delhi
Travel Date	Unknown
Number of Passengers	Unknown

6. Next Appropriate Question

Since **Travel Date** is the next missing required slot:

"What is your travel date?"

7. Final Explanation

Dialogue State Tracking stores information from previous turns and maintains it throughout the conversation. It prevents the user from repeating information and allows the agent to ask only for missing details. Therefore, it improves **coherence, consistency, and efficiency** in multi-turn conversations.